

Correct Decisions, Incorrect Diagnostic Reports (Supplemental)

Uddhav P. Gautam,¹ Bishnu Bidari,² Sajjan Lamichhane,³ Sahil Dudani,¹ Randy Marchany,¹ Haining Wang¹

¹Virginia Tech, Blacksburg, VA, USA ²Independent Researcher, Kathmandu, Nepal ³Independent Researcher, West Chester, PA, USA

AOSP source locators refer to frameworks/base at revision [94b4c163b7dfe5ce3607f7bb8456f9573f7de57d](#) (Android 17 r1; public tag android-17.0.0_r1). Every line number below was read at that exact revision. This supplement documents the source inspection, one derivation, and the controller-driven measurement reported in the paper: the inspected-source inventory, the constant-rate derivation behind Section 4.1 of the paper, the analytical counterexample’s call sequence, and the campaign’s configuration, protocol, timing bounds, and provenance. The analytical call sequence and the measured campaign are documented separately and use different configurations.

Inspected framework source

[apex/jobscheduler/service/java/com/android/server/job/controllers/JobStatus.java](#)

Item	Locator
Start map S (mLastTimesConstraintsUnsatisfied, keyed by public reason)	649–653
Total map T (mPendingJobReasonsStats, keyed by public reason)	654–658
History return limit (PENDING_JOB_HISTORY_RETURN_LIMIT=10)	646
Constraint setters relevant to the trace (setChargingConstraintSatisfied, setBatteryNotLowConstraintSatisfied, setTimingDelayConstraintSatisfied, setIdleConstraintSatisfied)	2134,2139,2154,2169
setConstraintSatisfied (state-change guard; eager stats update)	2262–2316 (guard 2263–2266; updater call 2313)
Constraint-to-public-reason mapping (constraintToPendingJobReason; requested vs. imposed)	2415–2487
Reason-list conversion (constraintsToPendingJobReasons; suppresses duplicate AS when several constraints map to it)	2491–2590 (explicit AS suppression 2543,2556)
Accounting updater (updatePendingJobReasonStats)	2688–2714 (onset overwrite 2695–2697; no-prior-start branch 2702–2704; total write 2712–2713)
Duration getter (getPendingJobReasonStats)	2721–2759 (job-age branch 2737–2740; T[r] + (q-S[r]) 2747–2751; missing-start warning 2753–2754)
hasConstraint (tracking predicate; includes dynamic constraints)	1953–1955
setStandbyBucket (Restricted entry; dynamic-constraint imposition)	1603–1615
Readiness calculation (duration maps unread)	2852–2986 (dynamic-constraint clause 2935–2938)
Reschedule clone of the stats map	994

[apex/jobscheduler/service/java/com/android/server/job/controllers/BatteryController.java](#)

Item	Locator
Calls setChargingConstraintSatisfied	82,85,204,207
Calls setBatteryNotLowConstraintSatisfied	89,212

[apex/jobscheduler/service/java/com/android/server/job/controllers/IdleController.java](#)

Item	Locator
Calls <code>setIdleConstraintSatisfied</code>	72,120

[apex/jobscheduler/framework/java/android/app/job/JobScheduler.java](#)

Item	Locator
PENDING_JOB_REASON_APP_STANDBY constant	163
<code>getPendingJobReasonStats</code> javadoc (lifetime aggregate; clearing; reboot; cross-reason overlap)	551–581
Cross-reference to the sibling history API	569
<code>@FlaggedApi (FLAG_GET_PENDING_JOB_REASON_STATS_API)</code> on the method	582–584

[apex/jobscheduler/service/java/com/android/server/job/JobSchedulerService.java](#)

Item	Locator
Service-side <code>getPendingJobReasonStats</code> (verbatim pass-through)	2292–2304
Binder stub (caller identity, namespace validation, and delegation; no duration recalculation)	5547–5557

[apex/jobscheduler/framework/java/android/app/JobSchedulerImpl.java](#)

Item	Locator
Client conversion (<code>Integer.parseInt</code> keys; <code>ParcelDuration.getDuration()</code> unwrap)	199–216 (conversion statement 203–212)

Inspected tests

[services/tests/mockingservicestests/src/com/android/server/job/controllers/JobStatusTest.java](#) — the seven tests whose names contain `PendingJobReasonStats`:

Test	Line
<code>testGetPendingJobReasonStats</code>	1713
<code>testDisabledFlagGetPendingJobReasonStats</code>	1757
<code>testGetPendingJobReasonStats_BatterySaver</code>	1799
<code>testGetPendingJobReasonStats_StillPending</code>	1825
<code>testDisabledFlagGetPendingJobReasonStats_StillPending</code>	1858
<code>testGetPendingJobReasonStats_correctlyAccumulates</code>	1890
<code>testDisabledFlagGetPendingJobReasonStats_correctlyAccumulates</code>	1937

The `correctlyAccumulates` test drives the scheduler’s `sElapsedRealtimeClock` through a `TestClock` advanced incrementally (`advanceTime(1000)` steps). The test assigns `enqueueTime` the clock value before the first advance, so the paper’s times for this test are ms since `enqueue`. The paper’s proposed middle query at 2500 corresponds to splitting the 2000–3000 advance into two 500-unit advances, preserving the final endpoint.

Correction commit

[c98cdae70252](#) — “Update certain logic for pending job reason stats.” (Varun Shah, January 2, 2026; bug 462813866). Changes the getter to include ongoing time after a condition becomes unmet again; copies the accounting maps on a rescheduling path; adds `testGetPendingJobReasonStats_correctlyAccumulates`. The correction leaves the updater’s shared-start overwrite unchanged; the pinned revision still contains that overwrite at lines 2695–2697.

Completed intervals can conceal missing ongoing time

This section gives the constant-rate generalization summarized in Section 4.1 of the main paper for the correction commit described above. All times and durations below are in ms. Let θ be the time of a processed event, and let T_θ be the value of the present stored total immediately after that event. Assume that this total correctly accounts for the contributing time through θ . Let the contribution rate ρ remain constant from then until query $q \geq \theta$. Here a rate of zero adds nothing and a rate of one counts each elapsed millisecond once; under the additive reference of the main paper, several unmet contributors to one reason give a rate above one. Correct accumulation gives $F(q) = T_\theta + \rho(q - \theta)$. A calculation omitting the ongoing interval returns $G(q) = T_\theta$. For nonnegative rates, these answers agree exactly when $q = \theta$ or $\rho = 0$. Consequently, queries at $q = \theta$ after processing that event can't distinguish these calculations. Neither can queries while $\rho = 0$, outside any waiting interval. A query before processing a closing event can distinguish them; event order matters. In the main paper's Table 2, the original queries occur after each charging interval closes. For the proposed query at 2500, $\theta = 2000$, $T_\theta = 1000$, $\rho = 1$, and $q = 2500$, so $F(q) = 1500$ while $G(q) = 1000$.

Counterexample call sequence (Table 1)

On one JobStatus for an ordinary, non-expedited job with requested minimum latency (unexpired beyond $e+33$), no deadline, no app-requested charging, battery-not-low, idle, or network constraint, and no constraint override; enqueue $eTime = e$ with $e \geq 0$ (the main paper takes $e = 0$); times are elapsed-clock ms relative to e . The listed setter timestamps are call arguments, while at each getter call `JobSchedulerService.sElapsedRealtimeClock.millis()` is taken to equal e plus the listed offset:

1. `setStandbyBucket(RESTRICTED_INDEX, ...)` — imposes the dynamic Restricted constraints.
2. At e : `setChargingConstraintSatisfied(e, true)`, `setBatteryNotLowConstraintSatisfied(e, true)`, `setIdleConstraintSatisfied(e, true)` — each takes the no-prior-start branch, adding $e - e = 0$; $T[APP_STANDBY]$ becomes present and zero; the start map stays empty.
3. $+10$: `setChargingConstraintSatisfied(e+10, false)` — start map gains $S[APP_STANDBY] = e+10$.
4. $+19$: `getPendingJobReasonStats()` → $APP_STANDBY$ duration 9.
5. $+20$: `getPendingJobReasonStats()` → 10; then `setBatteryNotLowConstraintSatisfied(e+20, false)` — overwrites $S[APP_STANDBY] = e+20$ with no accumulation into T .
6. $+21$: `getPendingJobReasonStats()` → 1 (both entries present; the missing-start warning's guard is false).
7. $+32$: `setChargingConstraintSatisfied(e+32, true)` — $S[APP_STANDBY] = e+20$ is present, so the branch accumulates $32 - 20 = 12$ into $T[APP_STANDBY]$ and deletes the start entry.
8. $+33$: `setBatteryNotLowConstraintSatisfied(e+33, true)` — the start map is now empty, so the no-prior-start branch adds job age $(e+33) - e = 33$, giving $T[APP_STANDBY] = 12 + 33 = 45$. No query is issued at either closing event.

After initialization, idle remains satisfied. The unexpired minimum latency alone keeps the job ineligible throughout the numbered sequence.

Controller-driven campaign

Purpose and terminology

This section documents the one-boot campaign reported in the main paper. It measures a public duration after two conditions sharing a reason become met again. Here C , BNL , and I denote charging, battery-not-low, and device-idle; AS abbreviates $APP_STANDBY$. Unless a tracker observation is explicitly identified, met and unmet refer to the state recorded for the job by JobScheduler. An *attempt* consists of setup, scheduling one job, two public duration queries separated by four battery-state interventions, and cleanup. The first query is the *baseline query*, before those interventions; the second is the *final query*, after them. A *campaign* is the retained sequence of attempts. The experiment reported here used one configured emulator image and one boot.

Platform, caller, and job

The campaign used a locally source-built AOSP emulator, not an official Google-distributed image. Its target was `sdk_phone64_x86_64`, with release configuration `cp2a`, `userdebug/test-keys`, and build ID `CP2A.260605.016`. The

framework’s `JobStatus.java` source file was byte-identical to its pinned Android 17 r1 counterpart. The sole source-tree change was a Goldfish product-overlay resource setting `config_enableAutoPowerModes` to true. In this runtime configuration, that override enabled the App Standby subsystem so the package could be assigned to the Restricted bucket. The same resource also enabled the system-wide light- and deep-idle modes; those modes are distinct from the job’s requested device-idle constraint. The override did not modify the Binder or client path.

A host-side Python harness used Android Debug Bridge (ADB) shell commands for setup, battery-state changes, and privileged observations. A separate probe app, requesting no permissions, scheduled its own job and queried its durations through the public `getPendingJobReasonStats(jobId)` API. The harness initiated those operations with ordered broadcasts to the app’s `ProbeReceiver`; the receiver executed under the installed app’s user ID. The receiver required senders to hold `android.permission.DUMP`, which the shell sender held. This requirement did not grant that permission to the app. For each query, the receiver serialized every returned reason-duration entry for the harness to collect.

The job used ID 17001, requested device idle and a 900,000-ms minimum latency, and requested neither charging, battery-not-low, nor a network. Restricted imposed the three dynamic constraints, but requested *I* mapped to its dedicated public reason. Only *C* and *BNL* exercised the shared *AS* path. The minimum-latency and device-idle constraints remained unmet through the final query; either was sufficient to keep the job ineligible. Compared with the analytical call sequence above, this job requested idle and its two closing transitions occurred in the reverse order.

Per-attempt sequence

Before scheduling, the harness initialized the simulated battery inputs, polled `JobScheduler`’s battery-state tracker until two consecutive rounds reported charging and battery-not-low, and assigned the app to Restricted. No corresponding job record existed during these initial mutations. After scheduling returned, the protocol included a planned 30-s wait. The harness then captured the job’s state immediately before and after the public baseline query. We call these captures A and B. The baseline had to report an explicitly present *AS* entry of zero while the job recorded both *C* and *BNL* as met; an absent entry did not qualify.

Between the public queries, the harness drove the following recorded state sequence. The last column lists the contributors recorded as unmet at each checkpoint under the mapping above; it is not an additional field exposed by the public stats API.

Checkpoint	<i>C</i>	<i>BNL</i>	Unmet contributors to <i>AS</i>
Baseline	Met	Met	None
After charging becomes unmet	Unmet	Met	<i>C</i>
After battery-not-low becomes unmet	Unmet	Unmet	<i>C, BNL</i>
After battery-not-low becomes met	Unmet	Met	<i>C</i>
After charging becomes met	Met	Met	None

Each intervention was a complete harness event group, not a single shell command. The charging groups each changed battery status and simulated charger-online state; the battery-not-low groups changed battery level. After each group, the harness waited for two matching tracker-polling rounds. The planned waits between successive groups were 2 s, 15 s, and 2 s, each starting after the preceding group completed. These waits do not specify the exact gaps between recorded job transitions: the framework’s `BatteryController` bookkeeping is asynchronous. The app made its final public query after the last group’s matching polls. The harness then took a third privileged job capture, performed the remaining job-state, configuration, integrity, and boot checks, and verified cleanup.

Qualification and retained cohort

Before acquisition, the campaign plan fixed a target of 30 protocol-qualified attempts, a cap of 40 attempts, and retention of every attempt. The harness checked the plan’s qualification list, selection rule, and source-hash binding against its configured constants before running. Qualification required the 20 plan-designated analysis predicates and the additional execution, schema, command-log, cleanup, and boot requirements. This decision is recorded as `protocol_qualified`. Eight further result checks were kept separate: comparisons among the returned *AS* value, the same-call minimum-latency value, the job-age and reference bounds defined below, and the updater predictions, and a check that

AS was the unique largest entry of the final map. Their truth values affected neither inclusion nor continuation. All eight held in all 31 retained attempts, including the one attempt excluded from qualification below.

The qualification checks included successful scheduling and the same app identity in the schedule and query results; the same nonempty scheduler-displayed `JobStatus` identity in captures A, B, and the final capture; and the prescribed requested and dynamic constraints. The probe’s final-query report recorded zero `onStartJob()` callbacks, and the subsequent job-state output had to report waiting or pending, not active or ready. The harness also checked agreement between its own computation of the final map’s maximum and the maximum and reason set reported by the probe.

For history validation, each entry was represented by an enqueue-relative timestamp offset and a satisfaction mask, the bit set recording which constraints were met. Captures A and B had to contain identical ordered (offset, mask) sequences. Starting from capture B’s last mask and keeping all other constraint bits unchanged, the harness formed the four expected masks shown in the checkpoint table. It appended them to capture B’s satisfaction-mask sequence. The final capture’s satisfaction-mask sequence had to equal the last ten masks of this constructed sequence, or all masks when fewer than ten. The final capture’s last four history entries were matched in order to the four interventions; their offsets had to increase strictly and pass the event-specific timing checks described next. These history requirements were qualification checks, not tests of the final duration’s correctness.

The campaign stopped after attempt 31 supplied the 30th qualifying trace. Attempt 15 was the only exclusion: its baseline *AS* entry was 2 ms, so it failed the present-zero requirement. It remains in the archive. All 31 attempts ran sequentially in one emulator process and one boot, launched with `-wipe-data` and `-no-snapshot`; all cleanup checks passed. This cohort measures within-boot repeatability on one configured image, not variation across independent boots or devices.

Clock observations and job-transition timestamps

This section describes the clock observations behind the timing checks just mentioned and the bounds of the next section; all equations here use ms. For attempt i , let e_i be the record’s enqueue timestamp and q_i the elapsed-realtime timestamp read by the duration getter during the final public query. Job age at that query is $q_i - e_i$. The app took elapsed-realtime readings immediately before and after its scheduling call and both public-query calls. The successful synchronous scheduling path assigned the enqueue timestamp before returning, so e_i lay within the app’s scheduling interval. Similarly, q_i lay within the app’s final-query interval.

The host separately recorded device-uptime readings around those operations and around each intervention. An intervention bracket began before its first battery mutation and ended after its second matching tracker-polling round. These polls observed `BatteryStateTracker`, not completion of the job-specific accounting update. The harness also obtained the job’s constraint history with `dumpsys jobscheduler`.

The plan fixed an endpoint allowance $\delta=20$ ms, accounting for the 10-ms grid of the archived `/proc/uptime` readings and possible residual differences between separately sampled clocks. The analysis subtracted δ from each raw uptime lower endpoint and added it to each raw upper endpoint. A qualification check required the app’s scheduling and query timestamps to lie, in order, inside their respective widened uptime brackets. Denote the widened scheduling and final-query brackets by $[s_i^-, s_i^+]$ and $[f_i^-, f_i^+]$. The resulting conservative job-age interval is

$$[f_i^- - s_i^+, f_i^+ - s_i^-].$$

Subtracting the app’s own scheduling interval from its final-query interval analogously gives the app-elapsed job-age interval.

For the following notation, $\neg C$ and $\neg BNL$ label recorded transitions to unmet, and C and BNL label transitions back to met. For a transition x , let $[\ell_i(x), u_i(x)]$ be its widened intervention bracket, and let $\tau_i(x)$ be its recorded timestamp minus e_i . The state-changing setter writes the same supplied timestamp to the constraint history and the duration updater. The privileged dump expresses both that timestamp and enqueue time relative to a common dump-time origin, preserving integer ms. Subtracting the parsed enqueue-time value from the parsed history-entry value recovers $\tau_i(x)$. Qualification required each such offset to lie in the difference interval

$$[\ell_i(x) - s_i^+, u_i(x) - s_i^-].$$

There was also a separate timestamp-containment verification, not a qualification gate. Adding $\tau_i(x)$ to both endpoints of the app’s elapsed-realtime scheduling interval (not the host’s uptime scheduling bracket) produces an interval containing

the timestamp supplied to the corresponding setter. In all 31 retained attempts, each resulting interval lay wholly inside its corresponding raw intervention bracket, even before widening by δ . This verified containment, not tracker propagation or difference-interval membership alone, justifies using the intervention brackets to bound the recorded job transitions.

Reference bounds and updater reconstruction

The reference window begins at enqueue and ends at the final query. The *union* reference counts an instant once when any constraint contributes to *AS*; the *additive* reference counts it once per contributing constraint. They are comparison rules, not claims about Android’s intended handling of overlap. In every qualified trace, capture B’s first met entries for *C* and *BNL* each had enqueue-relative offset zero. Their initially unmet periods therefore had zero length within the reference window under the recorded-timestamp convention. Requested *I* did not contribute to *AS*. The verified later *BNL*-unmet interval lay within the *C*-unmet interval. Since contribution can occur only during the corresponding recorded-unmet period, conservative upper bounds on the two reference durations are

$$U_i = u_i(C) - \ell_i(\neg C), \quad A_i = U_i + u_i(BNL) - \ell_i(\neg BNL).$$

The conservative job-age upper bound is $B_i = f_i^+ - s_i^-$. For a duration from a recorded event x to y , the corresponding interval lower endpoint is $\ell_i(y) - u_i(x)$; component interval bounds add by summing their respective endpoints.

At baseline, *C* and *BNL* were met, so *AS* was noncurrent. For a noncurrent reason, the getter includes an entry exactly when that key exists in the total store. The public $AS \mapsto 0$ baseline entry therefore establishes a present-zero stored total, not merely a zero computed for an active interval. The baseline query leaves that state unchanged.

Starting from this total, the charging unmet transition records the shared start, and the battery-not-low unmet transition replaces it. When battery-not-low becomes met, the updater credits $\tau_i(BNL) - \tau_i(\neg BNL)$ and removes the start. The later charging met transition then takes the enqueue fallback and adds $\tau_i(C)$, the job’s age at that transition. The exact-history prediction for the final return is therefore

$$\widehat{D}_{AS,i} = (\tau_i(BNL) - \tau_i(\neg BNL)) + \tau_i(C).$$

Since $\tau_i(C)$ already spans the credited *BNL* interval, $\widehat{D}_{AS,i}$ exceeds the job age at the final query by that interval less the gap between the charging met transition and the query; the planned 15-s wait between the battery-not-low groups therefore sets the approximate size of the excess reported below. At the final query, *AS* is again noncurrent, so the getter copies the stored total without a query-time addition. Bounding the same two updater additions using the widened timing brackets gives

$$\begin{aligned} P_i^- &= \ell_i(BNL) - u_i(\neg BNL) + \ell_i(C) - s_i^+, \\ P_i^+ &= u_i(BNL) - \ell_i(\neg BNL) + u_i(C) - s_i^-. \end{aligned}$$

The interval $[P_i^-, P_i^+]$ and point value $\widehat{D}_{AS,i}$ are source-guided predictions of the unchanged updater, not repaired reference values. Because the exact-history prediction uses timestamps written by the same setters that update the duration, matching it is not an independent test of that internal path. The reference comparisons serve a different purpose: a return above both U_i and A_i cannot equal either reference duration.

The plan’s separate 500-ms constant did not widen timing brackets. Checks requiring one value to exceed another needed a gap strictly greater than 500 ms. Checks requiring a value v to fit an interval $[L, U]$ accepted $L - 500 \leq v \leq U + 500$. The exact-history residual used integer millisecond equality, not that tolerance.

Results and scope

The table reports the 30 protocol-qualified attempts. All entries are seconds; each row gives the median, minimum, and maximum across that cohort. For every row reporting a difference, the difference was calculated within each attempt before summarizing. These are not confidence intervals or differences between marginal summaries.

Quantity	Median	Minimum	Maximum
Public AS duration	65.500	65.251	65.575
Public minimum-latency duration	50.495	50.252	50.556
Conservative job-age upper bound B_i	50.610	50.370	50.680
Union-reference upper bound U_i	20.415	20.170	20.460
Additive-reference upper bound A_i	36.000	35.750	36.070
Public AS duration minus public minimum-latency duration	15.001	14.985	15.034
Public AS duration minus B_i	14.884	14.848	14.916
Public AS duration minus $\widehat{D}_{AS,i}$	0.000	0.000	0.000

Minimum latency and requested device idle remained unmet from enqueue and had no stored totals, so the getter returned job age for both. Their final-query durations agreed exactly in all 31 retained attempts, including the excluded attempt. In every qualified attempt, the public minimum-latency duration lay inside the app-elapsed job-age interval.

In every qualified attempt, the final public AS value exceeded its conservative job-age upper bound by more than 14.8 s, exceeded both reference upper bounds, and equalled $\widehat{D}_{AS,i}$ to the millisecond. AS was no longer current, while minimum latency still prevented execution. Exceeding job age alone is not the test of both reference rules: additive contributor time can exceed elapsed time in other histories. Here the return also exceeds the separately bounded reference durations. Only the baseline and final public queries were made. The campaign establishes this post-closure over-count on one configured emulator; it does not measure prevalence on physical devices.

Archive identification and execution provenance

The campaign identifier is cp2a-r1-as-20260819T043340Z. The retained records include campaign-plan.json, campaign-result.json, the full SHA256SUMS manifest, and evidence1.json through evidence31.json. The result index binds each attempt by SHA-256. The plan identifies the harness/probe source set and expected build-output hashes. At launch and in every attempt, the six recorded runtime-artifact hashes matched their expected build-output hashes. The attempt field runtime_artifact_sha256_by_path agrees with the plan's build_output_runtime_artifact_sha256 for those paths, including services.jar, framework.jar, and both scheduling APEX jars. Each freshly installed probe APK also matched the retained APK's hash. These comparisons connect the observed calls to the stated build; they are separate from the source-file identity claim above.